



Baidu IOS端归因升级方案

更新时间：2026年 6 月

百度时代网络技术(北京)有限公司

目录	
方案一、百度归因.....	3
方案二、客户归因.....	3
1 第一优先级：IDFA.....	3
2 第二优先级：CAID.....	6
A) 开源规则 CAID.....	6
B) 中广协 CAID.....	8
3 第三优先级：模糊归因.....	8
客户须知.....	
附录一	10
附录二	13
附录三	24

当前文档主要介绍**IOS 端归因**升级方案，包含百度归因和客户归因两大方案。

方案一、百度归因

1. 百度归因的优势：百度通过 SDK 上报的多种 key（如 IDFA、IP、UA 等）分优先级的与投放数据进行拼接
2. 百度归因具体使用方法：应用 SDK 转化追踪工具是通过百度应用 SDK 能力上报转化类型且百度进行自归因的数据回传方式。使用详情如下：<http://ocpc.baidu.com/developer/d/guide/?iurl=app%2Fapp-doc%2Fsdk%2F>

方案二、客户归因

1 第一优先级：IDFA

1. 客户获取百度监测地址中的 IDFA 内容，目前通配符没有变化，客户不需要升级。

a) 百度信息流：

<http://afddocument.bj.bcebos.com/ocpc/%E7%99%BE%E5%BA%A6%E5%8E%9F%E7%94%9F%E4%B>
[F%](http://afddocument.bj.bcebos.com/ocpc/%E7%99%BE%E5%BA%A6%E5%8E%9F%E7%94%9F%E4%B)
[A1%E6%81%AF%E6%B5%81%E5%B9%BF%E5%91%8A%E8%BD%AC%E5%8C%96%E6%95%B0%E6%8D](http://afddocument.bj.bcebos.com/ocpc/%E7%99%BE%E5%BA%A6%E5%8E%9F%E7%94%9F%E4%B)
[%AEapi%E5%AF%B9%E6%8E%A5%E6%96%87%E6%A1%A3-%E6%AD%A3%E5%BC%8F%E7%89%88.pd](http://afddocument.bj.bcebos.com/ocpc/%E7%99%BE%E5%BA%A6%E5%8E%9F%E7%94%9F%E4%B)
[f](http://afddocument.bj.bcebos.com/ocpc/%E7%99%BE%E5%BA%A6%E5%8E%9F%E7%94%9F%E4%B)
[_](http://afddocument.bj.bcebos.com/ocpc/%E7%99%BE%E5%BA%A6%E5%8E%9F%E7%94%9F%E4%B)

idfa	{{IDFA}}	IOS 设备标识		IOS 设备标识：原值
------	----------	----------	--	-------------

b) 百度搜索

<http://ocpc.baidu.com/developer/d/guide/?iurl=app%2Fapp-interface%2FREADME1.html>

idfa	{{IDFA}}	IOS设备标识	原值，例：40879ADB-E3E8-4FD4-BE28-86DF97532DC7	无值时替换为NULL
------	----------	---------	---	------------

2. IDFA 值的变化：目前 IOS14 升级对 IDFA 无影响；到 21 年初 IOS14 系统升级之后对 IDFA 影响如下

- a) 系统升级后，用户从 IOS13 升级 IOS14，IDFA 值的变化
 - i. 若用户授权，获取新的 IDFA 的值，与之前IOS13 不一致
 - ii. 若用户不授权，获取到 ‘000000000-0000-0000-0000-000000000000’
- b) 系统升级后，IOS14 自身 IDFA 值的变化
 - i. 若之前是通过用户弹窗单独授权，则升级后获取新的 IDFA 的值与之前一致
 - ii. 若之前用户不是通过弹窗单独授权，则升级后获取新的 IDFA 的值与之前不一致

3. IDFA 归因注意项

- a) 客户侧（客户自己评估）：把系统升级前的 IDFA 存储，待系统升级后，若获取不到 IDFA，可用之前保存的 IDFA 进行归因。
- b) 百度侧：会把系统升级前的 IDFA 存储，待系统升级后，若获取不到IDFA，下发之前保存的IDFA

4. 客户获取 IDFA 的代码变化

a) 系统升级前

- i. 先导入框架

```
#import <AdSupport/ASIdentifierManager.h>
```

- ii. 获取广告标识

```
// 判断在设置-隐私里用户是否打开了广告跟踪
```

```
if ([[ASIdentifierManager sharedManager] isAdvertisingTrackingEnabled]) {
```

```
    NSString *idfa = [[ASIdentifierManager sharedManager].advertisingIdentifier UUIDString];
```

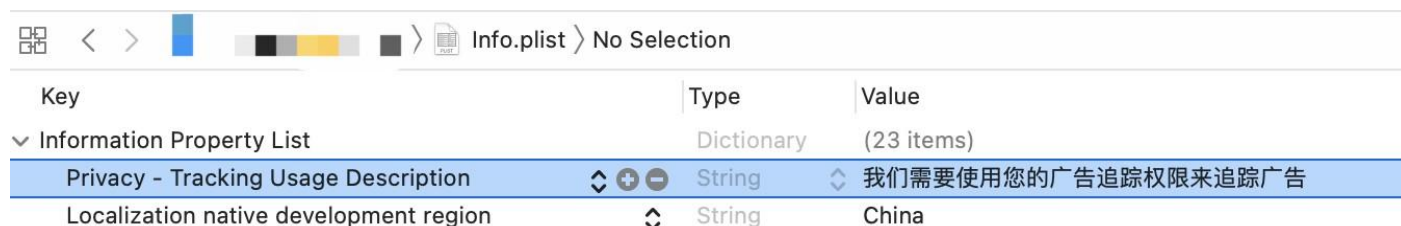
```
}
```

b) 系统升级后

i. 需要先请求跟踪权限，用户同意后才能获取到广告标识。

ii. 在 info.plist 文件里添加跟踪权限请求描述文字

注意：该权限只有xcode12 及以上版本才有，现在大家只能去下载xcode12 版本的来使用了。



Key	Type	Value
Information Property List	Dictionary	(23 items)
Privacy - Tracking Usage Description	String	我们需要使用您的广告追踪权限来追踪广告
Localization native development region	String	China

iii. 导入框架

```
#import <AppTrackingTransparency/AppTrackingTransparency.h>
```

```
#import <AdSupport/ASIdentifierManager.h>
```

iv. 获取 idfa

```
if (@available(iOS 14, *)) {
    // iOS14 及以上版本需要先请求权限
    [ATTrackingManager
    requestTrackingAuthorizationWithCompletionHandler:^(ATTrackingManagerAuthorizationS
    tatus status) {
        // 获取到权限后，依然使用老方法获取 idfa
        if (status == ATTrackingManagerAuthorizationStatusAuthorized) {
            NSString *idfa = [[ASIdentifierManager sharedManager].advertisingIdentifier
            UUIDString];
        }
    }
}
```

```
    }  
  
    } else {  
  
        // iOS14 以下版本依然使用老方法  
  
        // 判断在设置-隐私里用户是否打开了广告跟踪  
  
        if ([[ASIdentifierManager sharedManager] isAdvertisingTrackingEnabled])  
            { NSString *idfa = [[ASIdentifierManager sharedManager].advertisingIdentifier  
                UUIDString];  
  
            }  
  
    }  
}
```

2 第二优先级：CAID

CAID 目前有两个版本：开源规则生成的 CAID，中广协生成的 CAID，两者都可以用来作为归因的字段。客户可以自行选择

a) 开源规则 CAID

1. CAID 是什么？

a) CAID 组成：分为三个部分

- i. 第一部分：协议版本号，暂定为两位字符串，每位字符串暂定取值范围为 0-9 和 a-z (36 位)
- ii. 第二部分：由 5 个参数，硬盘大小+设备类型+内存大小+系统版本+系统更新时间以字符串"+"连接后做 MD5 摘要取得
- iii. 第三部分：由 4 个参数：用户名+运营商名+国家码+系统启动时间以字符串"+"连接后做 MD5 摘要取得

iv. 最终CAID 为以上三部分字符串由字符串"_"拼接后取得

b) CAID 举例：

00_209F3FAFDA2A6F7C674962CFAC7307CC_483556C8D01464E3551A19D88E56
AE2C

c) 归因方案：

i. 归因方式：建议只用 CAID 的第一部分和第二部分同时匹配；不使用第三部分

ii. 注：针对深度转化（次留、付费等），建议广告主先采用CAID 把激活与点击进行归因，然后用 IDFV（供应商标识符）把深度转化与激活进行归因。

2. 广告主如何获取百度媒体的 CAID

a) 百度信息流

i. 在监测地址中新增{{CAID}}或__CAID__ 通配符，参数说明：

参数名	通配符	描述	参数值	说明
caid	__CAID__ 或者 {{CAID}}	CAID(China Anonymization ID)	参见附录示例	取值会进行 urlencode，取值示例和规则说明请见 附录一 ；

ii. 现存转化追踪工具中的监测地址，可以通过对转化追踪工具进行编辑，在监测地址中新增该通配符：

百度信息流操作路径：工具-转化追踪工具-编辑

								输入转化名称关键字	自定义列	2020/8
☐	转化名称	转化追踪方式	创建时间	APP类型	转化应用	转化配置状态	转化目标	分享	操作	
☐	trans_362393	应用API-下载	2020-09-03 10:29:57	Android	Demo3	已激活	激活	—	编辑删除查看重新关联	

b) 百度搜索

i. 在监测地址中增加{{CAID}}通配符，参数说明如下：

参数名	通配符	描述	说明	备注
caid	{{CAID}}	CAID(China Anonymization ID)	取值示例和规则说明请见 附录一	无值时替换为 NULL

ii. 现存应用API 转化追踪中的监测地址，可以通过对转化追踪工具进行编辑，在监测地址中新增该通配符；（编辑监测地址建议重新联调）

百度搜索操作路径：工具中心-转化追踪-编辑

转化名称	操作	监测范围	转化配置状态	追踪目标	转化类型	接入方式
☐ 测试222	详情 编辑 联调 删除	1个计划	☒ 无需联调	应用	其他	应用API - 仅Android

3. 广告如何自己生成CAID 内容。详情见**附录二**

b) 中广协 CAID

中广协 CAID 方案说明可参照【互联网广告技术实验室开发者平台】<http://www.cnaa123.com/caid/login>

- 按照中广协要求，2026年度更新算法计划于6月22日（周一）开始升级中心服务器，支持新版本算法，6月30日（周二）完成正式切换。升级期间，建议优先使用新版本（20260506），旧版本（20250325）做为保底版本，便于平滑切换。
- **20230330版本的caid将不再返回**

百度侧将通过应用API 转化追踪的监测地址，以通配符形式下发中广协 CAID ；（取值示例 规则说明见**附录一**）

3 第三优先级：模糊归因

1. 模糊归因是什么？

- 概念：模糊归因是通过 ip、ua 等可以公开对外的相关非完全精准的有效信息进行归因的方式。
- 方案：有两种归因方案

	使用字段	方式	备注
方案一	IP+UA（主要是从 UA 中获取系统版本）， 两者同时匹配	点击归因，建议窗口期点 击-激活24h内	相对方案一，方案二的准 确率有所提升

方案二	IP+UA（主要是从 UA 中获取系统版本） +机型（从百度{{DEVICE_INFO}}通配符 中获取），三者同时匹配		
-----	--	--	--

注：针对深度转化（次留、付费等），建议广告主先采用模糊归因把激活与点击进行归因，然后用 IDFV（供应商标识符）把深度转化与激活进行归因。

2. 广告主如何获取百度媒体 IP、UA、机型内容

a) 百度信息流

- i. 在监测地址中新增{{IP}}或__IP__、{{UA}}或__UA__、{{DEVICE_INFO}}或__DEVICE_INFO__通配符，具体文档：

<http://afddocument.bj.bcebos.com/ocpc/%E7%99%BE%E5%BA%A6%E5%8E%9F%E7%94%9F%E4%BF%A1%E6%81%AF%E6%B5%81%E5%B9%BF%E5%91%8A%E8%BD%AC%E5%8C%96%E6%95%B0%E6%8D%AEapi%E5%AF%B9%E6%8E%A5%E6%96%87%E6%A1%A3-%E6%AD%A3%E5%BC%8F%E7%89%88.pdf>

- ii. 现存转化追踪工具中的监测地址，可以通过对转化追踪工具进行编辑，在监测地址中新增该通配符：

百度信息流操作路径：工具-转化追踪工具-编辑

输入转化名称关键字 自定义列 2020/8									
☐	转化名称	转化追踪方式	创建时间	APP类型	转化应用	转化配置状态 ①	转化目标	分享	操作
☐	trans_362393	应用API-下载	2020-09-03 10:29:57	Android	Demo3	已激活	激活	—	编辑 删除 查看 重新联调

b) 百度搜索

- i. 在监测地址中新增{{IP}}、{{UA}}、{{DEVICE_INFO}}通配符，具体文档及参数说明：

<http://ocpc.baidu.com/developer/d/guide/?iurl=app%2Fapp-interface%2FREADME1.html>

参数名	通配符	描述	说明	备注
ip	{{IP}}	用户终端的公网 IP 地址	例：61.158.146.195 或 240e:398:1c90:9d00:5513:b9c9:650a:d9d	无值时替换为NULL

参数名	通配符	描述	说明	备注
ua	{{UA}}	数据上报终端设备 User Agent (建议从中分离出系统版本用于模糊归因 样例中标黄处)	例: Mozilla/5.0 (iPhone; CPU iPhone OS 14_4_2 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Mobile/15E148 swan/2.26.0 swan- baiduboxapp/12.6.5.10 baiduboxapp/12.6.5.10 (Baidu; P2 14.4.2)	无值时替换为NULL
device_info	{{DEVICE_INFO}}	设备信息	例: iPhone10,6	无值时替换为NULL

- ii. 现存应用API 转化追踪中的监测地址，可以通过对转化追踪工具进行编辑，在监测地址中新增该通配符；
(编辑监测地址建议重新联调)

百度搜索操作路径：工具中心-转化追踪-编辑

转化名称	操作	监测范围	转化配置状态	追踪目标	转化类型	接入方式
☐ 测试222	详情 编辑 联调 删除	1个计划	● 无需联调	应用	其他	应用API - 仅Android

3. 广告主自行获取 UA 和机型的方式，可详情见附录三

附录一

1. {{CAID}}取值示例- (百度搜索和百度信息流一致，下划线为通配符升级部分， **20230330版本的caid将不再返回**)
caid= %5B%7B%22caid%22%3A%5B%7B%22caid%22%3A%2200_D74F8140A32FBEC6125D43D6A6137E00_831581DD17DECAA652DA3E8516B2615C%22%2C%22version%22%3A%2200%22%7D%5D%2C%22generateTime%22%3A%221782201583%22%2C%22vendor%22%3A%220%22%7D%2C%7B%22caid%22%3A%5B%7B%22caid%22%3A%22c9435ec7f25d94e52588910314cf18a9%22%2C%22version%22%3A%2220260506%22%7D%2C%7B%22caid%22%3A%227c020cdd54552135bc1cf0bd462adf44%22%2C%22version%22%3A%2220250325%22%7D%2C%7B%22caid%22%3A%228c020cdd54552135bc1cf0bd462adf88%22%2C%22version%22%3A%2220230330%22%7D%5D%2C%22generateTime%22%3A

%221782201583%22%2C%22vendor%22%3A%221%22%7D%5D

urldecode 后结果（下划线为通配符升级部分）：

```
caid=[{"caid":{"caid":"00_D74F8140A32FBEC6125D43D6A6137E00_831581DD17DECAA652DA3E8516B2615C","version":"00"}}, {"generateTime":"1782201583","vendor":"0"}, {"caid":{"caid":"c9435ec7f25d94e52588910314cf18a9","version":"20260506"}}, {"caid":"7c020cdd54552135bc1cf0bd462adf44","version":"20250325"}], {"caid":"8c020cdd54552135bc1cf0bd462adf88","version":"20230330"}], {"generateTime":"1782201583","vendor":"1"}]
```

2. {{CAID}}数据结构说

明 [

{

"vendor":"0", //0 代表百度规则生成的CAID； 1 代表中广协规则生成的 CAID

"generateTime":"1652759174", //生成 CAID 的时间

"caid":[

{

"version":"01", //版本号，在生成规则升级期间标识为新版本的CAID

"caid":"BFB405DC6C3B16E8B1BA40BCD3A7C7C4" //说明：该值为 mock 数

据

},

{

"version":"00", //版本号，在生成规则升级期间标识为老版本的 CAID；在生成规则未升级期间，使

用该版本 CAID 即可

```
"caid":"00_DBE004E8793AB1A7D42E19DF646A364E_D6D5C80AB05EDCFFB98301C15711057A"

    }

]

},

{

    "vendor":"1", //0 代表百度规则生成的CAID; 1 代表中广协规则生成的 CAID

    "generateTime":"1605515747", //生成 CAID 的时间

    "caid":[

        {

            "version":"20230330", //版本号, 在生成规则升级期间标识为新版本的CAID,

            搜索和信息流同时下发20230330和20220111两个版本的caid;

            "caid":"fba39b7d459422ba65462e2f1a2c03d2" //说明: 该值为 mock 数

            据

        },

        {

            "version":"20220111", //版本号, 在生成规则升级期间标识为老版本的 CAID; 在生成规则未升

            级期间, 使用该版本 CAID 即可, 搜索和信息流同时下发20230330和20220111两个版本的caid

            "caid":"fd2ce8eb45c1502797f69704084cebc7" //说明: 该值为 mock 数据

        }

    ]

}
```

```
}
```

```
]
```

附录二

1. 说明

CAID 的生成，不采集隐私数据，仅传输加密后的结果，且加密结果不可逆，可有效保护终端用户的隐私和数据安全。

附录二将介绍如果获取CAID 生成所需的参数，及CAID 生成的规则。本文档适用于 iOS 8.0 及以上操作系统。

2. CAID 生成规则

- a) CAID 组成为三个部分: 第一部分，协议版本号，暂定为两位字符串，每位字符串暂定取值范围为 0-9 和 a-z (36 位)。
- b) 第二部分：由 5 个参数，硬盘大小+设备类型+内存大小+系统版本+系统更新时间以字符串"++"连接后做 MD5 摘要取得
- c) 第三部分：由 4 个参数，用户名+运营商名+国家码+系统启动时间以字符串"++"连接后做 MD5 摘要取得
- d) 最终CAID 为以上三部分字符串由字符串"_"拼接后取得

3. CAID 示例

00_209F3FAFDA2A6F7C674962CFAC7307CC_483556C8D01464E3551A19D88E56
AE2C

4. 相关参数获取

a) 所需头文件

```
#import <Foundation/Foundation.h>
```

```
#import <sys/sysctl.h>
```

```
#import <UIKit/UIDevice.h>
```

```
#import <CoreTelephony/CTTelephonyNetworkInfo.h>
```

```
#import <CoreTelephony/CTCarrier.h>
```

```
#import <CommonCrypto/CommonDigest.h>
```

b) MD5 方法

```
+ (NSString *)MD5:(NSString *)source {  
    if (![self isValidString:source] || source.length == 0)  
        { return nil;  
    }  
  
    const char *cStr = [source UTF8String];  
    if (cStr == NULL) {  
        return nil;  
    }  
  
    unsigned char digest[CC_MD5_DIGEST_LENGTH];  
    CC_MD5(cStr, (CC_LONG)strlen(cStr), digest);  
    NSMutableString *result = [NSMutableString stringWithCapacity:CC_MD5_DIGEST_LENGTH  
* 2];  
    for (int i = 0; i < CC_MD5_DIGEST_LENGTH; i++)  
        { [result appendFormat:@"%02X", digest[i]];  
    }  
  
    return result;  
}
```

c) 硬盘大小

```
+ (NSString *)diskSize  
    { int64_t space = -1;  
    NSError *error = nil;
```

```
NSDictionary *attrs = [NSFileManager defaultManager
attributesOfFileSystemForPath:NSHomeDirectory() error:&error];

if (!error) {

    space = [[attrs objectForKey:NSFileSystemSize] longLongValue];

}

if (space < 0)

    { space = -1;

}

return [NSString stringWithFormat:@"%lld", space];

}
```

d) 设备类型

```
+ (NSString *)systemHardwareByName:(const char *)typeSpecifier

{ size_t size;

sysctlbyname(typeSpecifier, NULL, &size, NULL, 0);

char *name = malloc(size);

sysctlbyname(typeSpecifier, name, &size, NULL, 0);

NSString *results = [NSString stringWithUTF8String:name];

free(name);

return results;

}

+ (NSString *)machine {

    NSString *machine = [self systemHardwareByName:"hw.machine"];
```

```
return machine == nil ? @"": machine;
```

```
}
```

e) 内存大小

```
+ (NSString *)memory {
```

```
    return [NSString stringWithFormat:@"%lld", [NSProcessInfo processInfo].physicalMemory];
```

```
}
```

f) 系统版本

```
+ (NSString *)systemVersion {
```

```
    return [UIDevice.currentDevice.systemVersion];
```

```
}
```

g) 系统更新时间

```
// system update time
```

```
+ (NSString *)getSysU
```

```
{ NSString *result = nil;
```

```
    NSString *information =
```

```
@"L3Zhci9tb2JpbGUvTGlicmFyeS9Vc2VyQ29uZmlndXJhdGlvbIByb2ZpbGVzL1B1YmxpY0luZm8vTUNNZXRhLnBsaXN0";
```

```
    NSData *data=[[NSData alloc] initWithBase64EncodedString:information options:0];
```

```
    NSString *dataString = [[NSString alloc] initWithData:data encoding:NSUTF8StringEncoding];
```

```
    NSError *error = nil;
```

```
NSDictionary *fileAttributes = [[NSFileManager defaultManager]
attributesOfItemAtPath:dataString error:&error];

if (fileAttributes) {

    id singleAttribute = [fileAttributes objectForKey:NSFileCreationDate]; if
    ([singleAttribute isKindOfClass:[NSDate class]]) {

        NSDate *dataDate = singleAttribute;

        result = [NSString stringWithFormat:@"%%.6f", [dataDate timeIntervalSince1970]];

    }

}

return result ? : @"";

}
```

h) 用户名

```
+ (NSString *)deviceName {

    NSString *result = [NSString stringWithFormat:@"%s", [UIDevice currentDevice].name];

    return [self MD5:result ? : @"unknown"];

}
```

i) 运营商名称

```
+ (NSString *)carrierInfo {

    #if TARGET_IPHONE_SIMULATOR

        return @"SIMULATOR";

    #else
```

```
static dispatch_queue_t _queue;

static dispatch_once_t once;

dispatch_once(&once, ^{
    _queue = dispatch_queue_create([NSString stringWithFormat:@"com.carr.%@", self]
UTF8String], NULL);
});

__block NSString *carr = nil;

dispatch_semaphore_t semaphore = dispatch_semaphore_create(0);

dispatch_async(_queue, ^(){
    CTTelephonyNetworkInfo *info = [[CTTelephonyNetworkInfo alloc] init];

    CTCarrier *carrier = nil;

    if ([[UIDevice currentDevice] systemVersion] floatValue) >= 12.1) {

        if ([info respondsToSelector:@selector(serviceSubscriberCellularProviders)])
        { #pragma clang diagnostic push
#pragma clang diagnostic ignored "-Wunguarded-availability-new"

            NSArray *carrierKeysArray = [info.serviceSubscriberCellularProviders.allKeys
sortedArrayUsingSelector:@selector(compare:)];

            carrier = info.serviceSubscriberCellularProviders[carrierKeysArray.firstObject];

            if (!carrier.mobileNetworkCode) {

                carrier = info.serviceSubscriberCellularProviders[carrierKeysArray.lastObject];

            }

#pragma clang diagnostic pop
        }
    }
}
```

```
    }  
}  
  
if (!carrier) {  
  
#pragma clang diagnostic push  
  
#pragma clang diagnostic ignored "-Wdeprecated-declarations"  
  
    carrier = info.subscriberCellularProvider;  
  
#pragma clang diagnostic pop  
  
}  
  
if (carrier) {  
  
    NSString *networkCode = [carrier mobileNetworkCode];  
  
    NSString *countryCode = [carrier mobileCountryCode];  
  
  
    if (countryCode && [countryCode isEqualToString:@"460"] && networkCode) {  
  
        if ([networkCode isEqualToString:@"00"] || [networkCode isEqualToString:@"02"] ||  
[networkCode isEqualToString:@"07"] || [networkCode isEqualToString:@"08"]) {  
  
            carr = @"中国移动";  
  
        }  
  
        if ([networkCode isEqualToString:@"01"] || [networkCode isEqualToString:@"06"] ||  
[networkCode isEqualToString:@"09"]) {  
  
            carr = @"中国联通";  
  
        }  
  
    }  
  
}
```

```
        if ([networkCode isEqualToString:@"03"] || [networkCode isEqualToString:@"05"] ||
[networkCode isEqualToString:@"11"]) {

            carr = @"中国电信";

        }

        if ([networkCode isEqualToString:@"04"])

            { carr = @"中国卫通";

        }

        if ([networkCode isEqualToString:@"20"])

            { carr = @"中国铁通";

        }

    } else {

        carr = [carrier.carrierName copy];

    }

}

if (carr.length <= 0)

    { carr = @"unknown";

    }

dispatch_semaphore_signal(semaphore);

});

dispatch_time_t t = dispatch_time(DISPATCH_TIME_NOW, 0.5 * NSEC_PER_SEC);

dispatch_semaphore_wait(semaphore, t);

return [carr copy];
```

```
#endif
```

```
}
```

j) 国家码

```
+ (NSString *)countryCode {
```

```
    return [[NSLocale currentLocale] objectForKey:NSLocaleCountryCode];
```

```
}
```

k) 系统启动时间

```
//system boot time
```

```
+ (NSString *)getSysB
```

```
{ struct timeval boottime;
```

```
int mib[2] = {CTL_KERN, KERN_BOOTTIME};
```

```
size_t size = sizeof(boottime);
```

```
time_t uptime = -1;
```

```
if (sysctl(mib, 2, &boottime, &size, NULL, 0) != -1 && boottime.tv_sec != 0)
```

```
{ uptime = boottime.tv_sec;
```

```
}
```

```
return [NSString stringWithFormat:@"%ld", uptime];
```

```
}
```

5. CAID 生成代码

a) 本地生成

```
#define CAID_VERSION @"00"
```

```
+(NSString *)currentCAID {  
    NSString *totalString = [self getStringTotal];  
    return [NSString stringWithFormat:@"%@_%@", CAID_VERSION, totalString];  
}  
  
+(NSString *)getStringTotal  
{  
    NSString *diskSize = [self  
        diskSize];  
    NSString *machine = [self machine];  
    NSString *memory = [self memory];  
    NSString *sysV = [self systemVersion];  
    NSString *sysU = [self getSysU];  
    NSString *source1 = [NSString stringWithFormat:@"%@++%@++%@++%@++%@", diskSize,  
machine, memory, sysV, sysU];  
    NSString *param1 = [self MD5:source1 ? source1 : @"unknown"];  
    NSString *deviceName = [self deviceName];  
    NSString *carrier = [self carrierInfo];  
    NSString *countryCode = [self countryCode];  
    NSString *sysB = [self getSysB];  
    NSString *source2 = [NSString stringWithFormat:@"%@++%@++%@++%@",  
deviceName, carrier, countryCode, sysB];  
    NSString *param2 = [self MD5:source2 ? source2 : @"unknown"];  
    NSString *result = [NSString stringWithFormat:@"%@_%@", param1, param2]; return  
    result;  
}
```

}

b) 服务器生成

若使用服务器生成，格式如下：

```
CAID_VERSION_MD5(diskSize++machine++memory++systemVersion++getSysU)_MD5(deviceName++carrierInfo++countryCode++getSysB)
```

注：格式中 MD5() 函数中操作对象均为字符串，格式中 "_", "++" 为不同参数间分隔符，类型为字符串。

附录三

1. UA 代码

a) iOS UA 生成代码

```
static WKWebView *webView = nil;

static dispatch_once_t onceToken;

dispatch_once(&onceToken, ^{
    webView = [[WKWebView alloc] initWithFrame:CGRectZero];
});

__block NSString *ua = nil;

[webView evaluateJavaScript:@"navigator.userAgent" completionHandler:^(NSString * _Nullable userAgent, NSError * _Nullable error) {
    ua = userAgent;
}];
```

b) 获取的 ua 示例：

Mozilla/5.0 (iPhone; CPU iPhone OS 13_3_1 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Mobile/15E148

2. 机型代码

a) iOS 机型获取代码

```
+ (NSString *)systemHardwareByName:(const char *)typeSpecifier
{
    size_t size;
    sysctlbyname(typeSpecifier, NULL, &size, NULL, 0);
    char *name = malloc(size);
    sysctlbyname(typeSpecifier, name, &size, NULL, 0);
    NSString *results = [NSString stringWithUTF8String:name];
    free(name);
    return results;
}

+ (NSString *)machine {
    NSString *machine = [self systemHardwareByName:"hw.machine"];
    return machine == nil ? @"": machine;
}
```